

École nationale supérieure en systèmes avancés et réseaux

Modélisation – Partie I

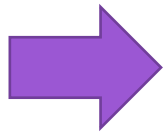
Stéphanie CHOLLET



Rappel : Complexité des logiciels

- Problématique du domaine
- Processus de développement
- Flexibilité inhérente du logiciel

- Conséquences :
 - Le risque de déficiences graves est élevé
 - La mise au point est lente et chaotique
 - La maintenance est disproportionnée
 - Le coût est astronomique

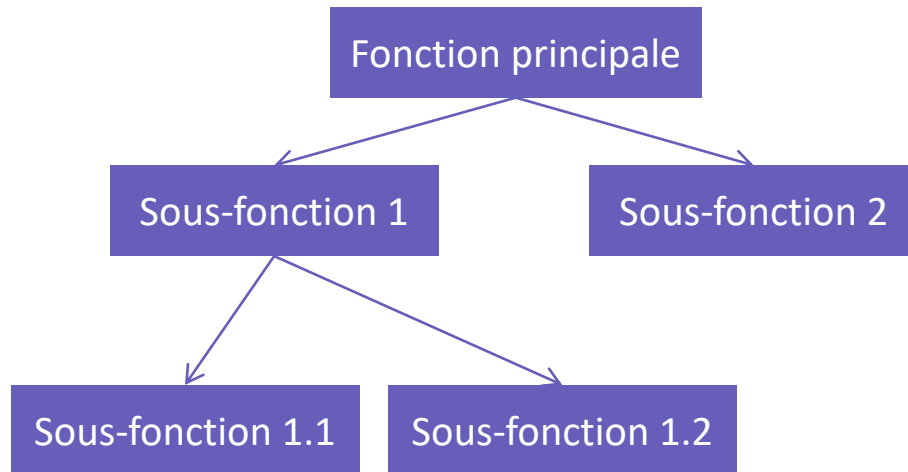


A défaut de réduire la complexité, il faut la maîtriser !

- « Diviser pour régner »
 - Raffinage récursif jusqu'à l'obtention d'éléments compréhensibles

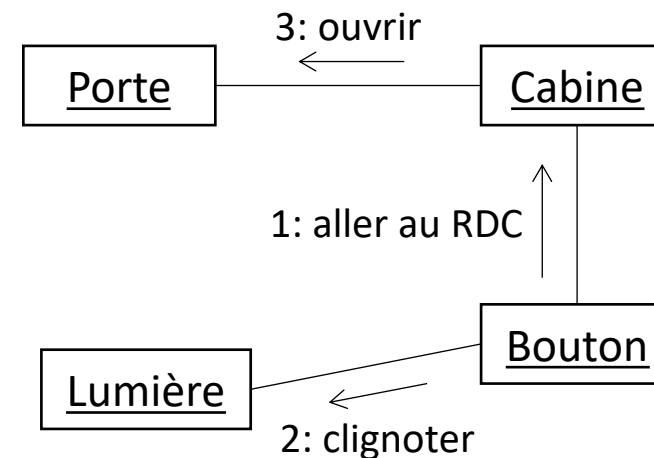
Décomposition algorithmique

- ▶ Approche traditionnelle
- ▶ Chaque module représente une étape du processus global
- ▶ Décomposition fonctionnelle depuis le cahier des charges jusqu'aux sous-programmes



Décomposition objet

- ▶ Approche plus récente
- ▶ Chaque module représente un objet du domaine de l'application
- ▶ Les objets sont des entités autonomes qui collaborent afin de réaliser un projet global

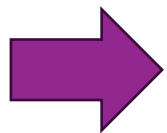


Comparaison des approches

- Approche fonctionnelle :
 - Semble intuitive
 - Mise en avant du « FAIRE »
 - Bien adaptée lorsque tout est connu
- MAIS :
 - Architecture rigide
 - Extension difficile
 - Peu adaptée à la découverte
- Approche objet :
 - Mise en avant de l' « ETRE »
 - Simple (petit nombre de concepts)
 - Raisonnement par abstraction (objets du domaine)
 - Adapté pour l'exploration et l'évolution
- MAIS :
 - Déroutant pour les habitués de la démarche fonctionnelle

Avantages de l'objet

- Conduit à des modèles plus stables
 - Basés sur le monde réel
- Structure indépendante des fonctions
 - Evolutivité
- Encapsule la complexité
 - Facilite la réutilisation



L'approche objet gère plus efficacement la complexité que l'approche fonctionnelle (réutilisabilité, évolutivité, stabilité)

La modélisation

Pourquoi modéliser ?

- Permettre une **meilleure compréhension** du monde réel
 - Systèmes réels trop complexes
 - Abstraction des aspects cruciaux du problème
 - Omission des détails
- Exemple : Qu'est ce qu'un terrain de football ?

Stade Vélodrome (Marseille)



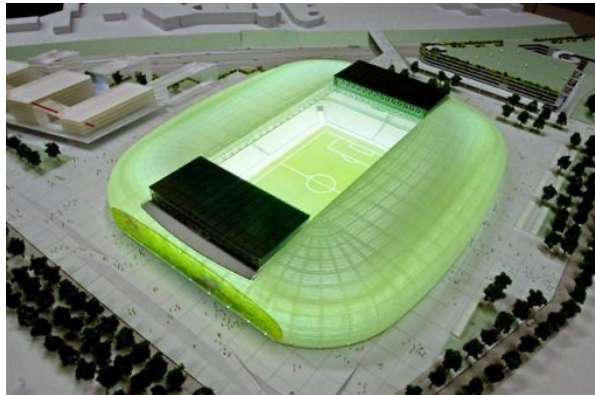
Parc des Princes (Paris)



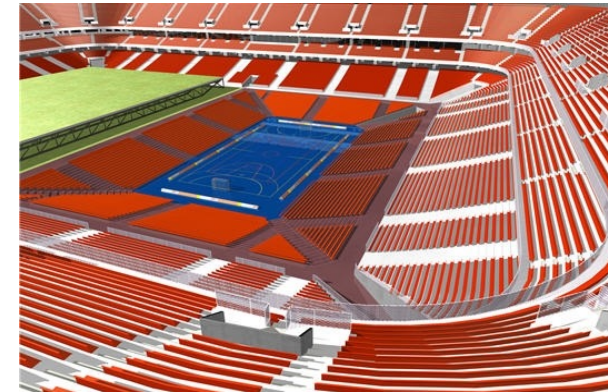
Pourquoi modéliser ?

- Permettre une **conception progressive**
 - Abstractions et raffinements successifs
 - Découpage en modules ou en vues
 - Génération des structures de données et des traitements
- Exemple : Conception par étape d'un stade

Maquette du stade de Lille



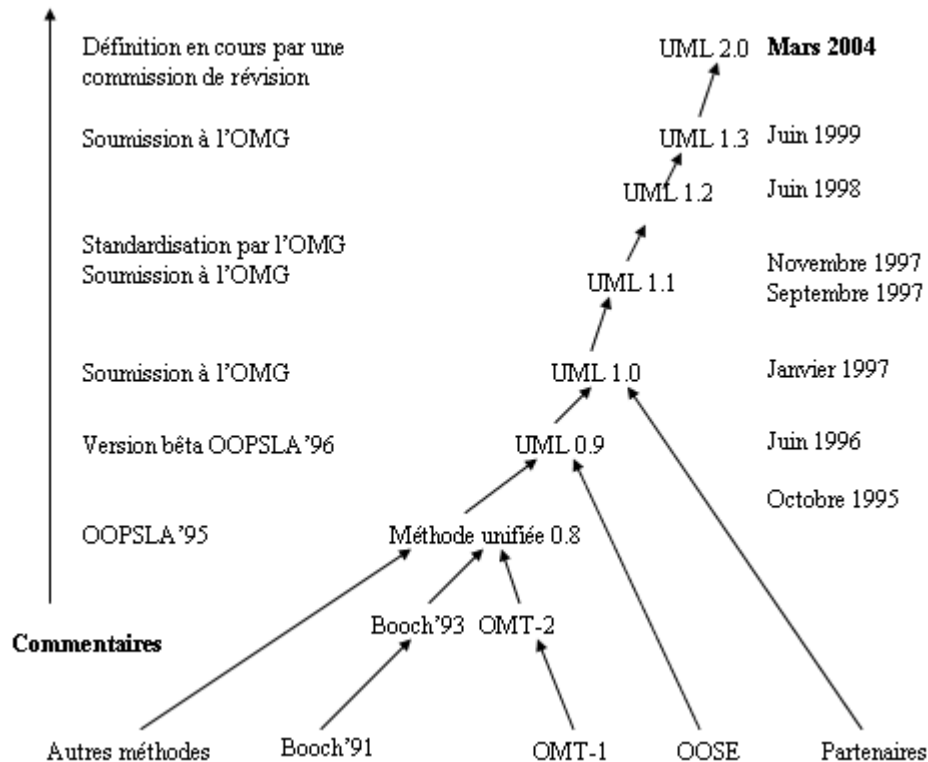
Plan des gradins du stade de Lille



De quoi a-t-on besoin pour modéliser ?

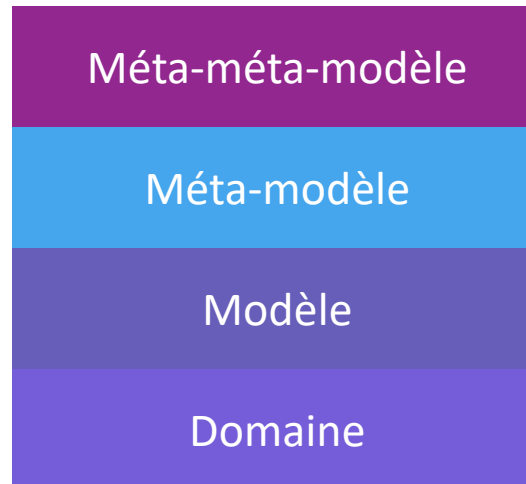
- Un langage de modélisation
 - Avec une notation claire
 - Avec une sémantique précise
- Caractéristiques d'un langage de modélisation :
 - Générique
 - Expressif
 - Flexible (configurable, extensible)
 - Syntaxe et sémantique non ambiguës

- Notations utilisées pour des modélisations orientées objets
- Notations unifiées basées sur de nombreuses méthodes



Objectifs de UML

- Représenter des systèmes entiers
- Etablir un couplage explicite entre les concepts et les artefacts exécutables
- Prendre en compte les facteurs d'échelle
- Créer un langage de modélisation utilisable à la fois par les humains et les machines



- Pas une notation **MAIS** des notations UML :
 - Diagramme des cas d'utilisation
 - Diagramme de classes
 - Diagramme d'objets
 - Diagramme de séquence
 - Diagramme de collaboration
 - Diagramme d'états
 - Diagramme d'activités
 - Diagramme de composants
 - Diagrammes de déploiement
 - Langages de contraintes (OCL)
 - Langage d'actions
 - Langage textuel
 - ...

Classement des diagrammes

- Diagrammes structurels :

- Diagramme de classes
- Diagramme d'objets
- Diagramme de déploiement
- Diagramme de composants

Vue statique

- Diagrammes comportementaux :

- Diagramme de cas d'utilisation
- Diagramme de séquence
- Diagramme d'activité
- Diagramme d'états-transitions
- Diagramme de collaboration

Vue dynamique

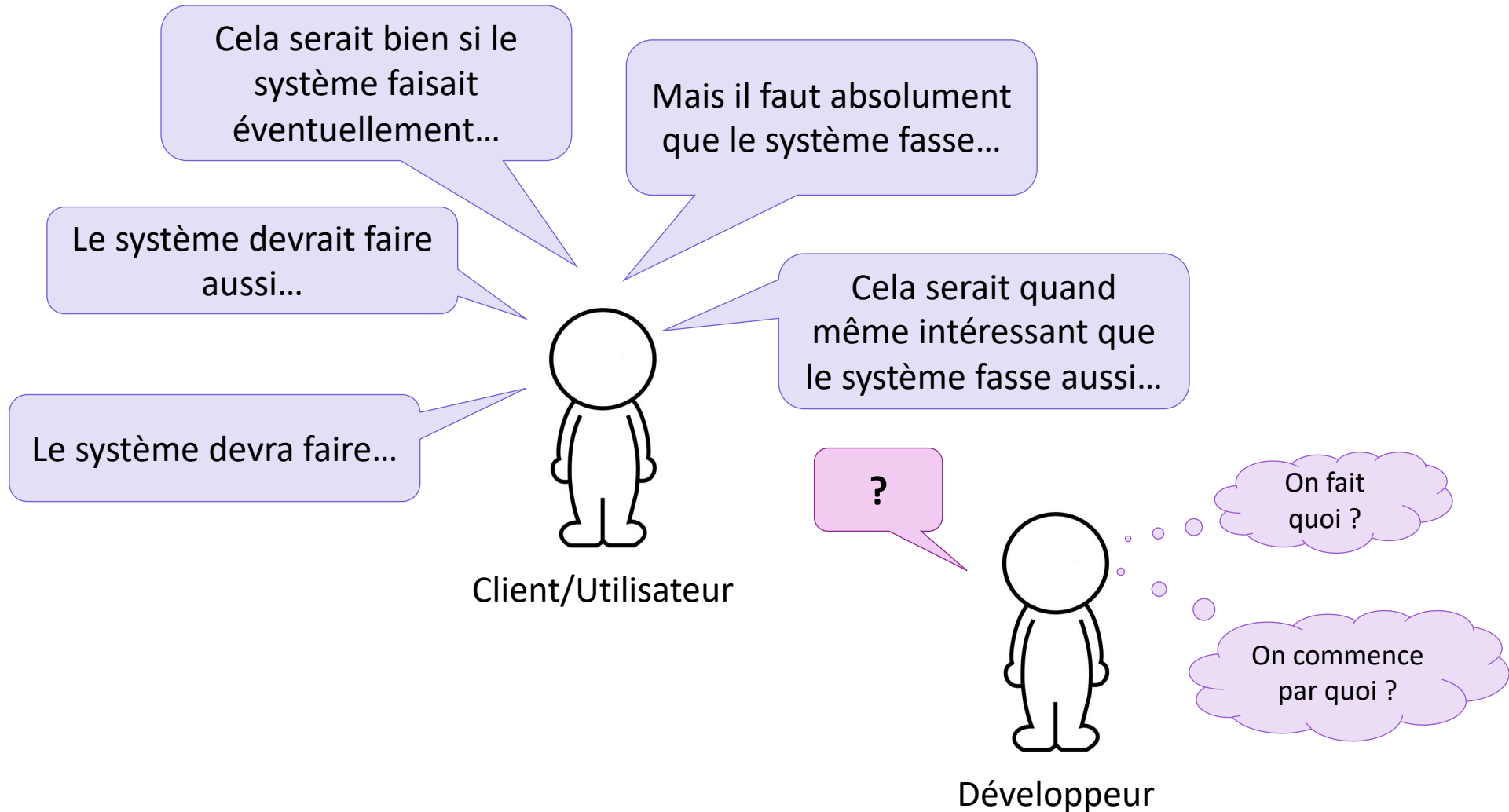
- La plupart des diagrammes se présentent sous la forme de graphes, composés de sommets et d'arcs
- Les diagrammes contiennent des éléments de visualisation qui représentent des éléments de modélisation
 - Les diagrammes peuvent montrer tout ou partie des caractéristiques des éléments de modélisation selon le niveau de détail utile dans le contexte d'un diagramme donné
 - En général, un diagramme représente un aspect ou un point de vue particulier

- Pour l'analyse et la conception :
 - Nécessaires pour comprendre le besoin
 - Nécessaires pour définir l'application : son comportement et les données qu'elle manipule
 - Modèles utilisés : Diagrammes de cas d'utilisation, de séquence, de classes, d'objets, d'états-transitions...
- Pour l'implantation :
 - Pour définir la structure de l'application
 - Modèles utilisés : Diagrammes de classes raffinés, de composants, de déploiement, de collaboration, d'activités...

Diagramme de cas d'utilisation

Modélisation des interactions

Mais que faisons-nous ?



- Formalisé par Ivar Jacobson
- Objectifs :
 - Décrire, sous la forme d'actions et de réactions, le comportement d'un système du point de vue d'un utilisateur
 - Définir les limites précises du système
- Un cas d'utilisation est :
 - Une manière spécifique d'utiliser le système
 - L'image d'une fonctionnalité du système, déclenchée en réponse à la stimulation d'un acteur externe

Exemple de diagramme de cas d'utilisation

- Représente les fonctions du système d'un point de vue des utilisateurs

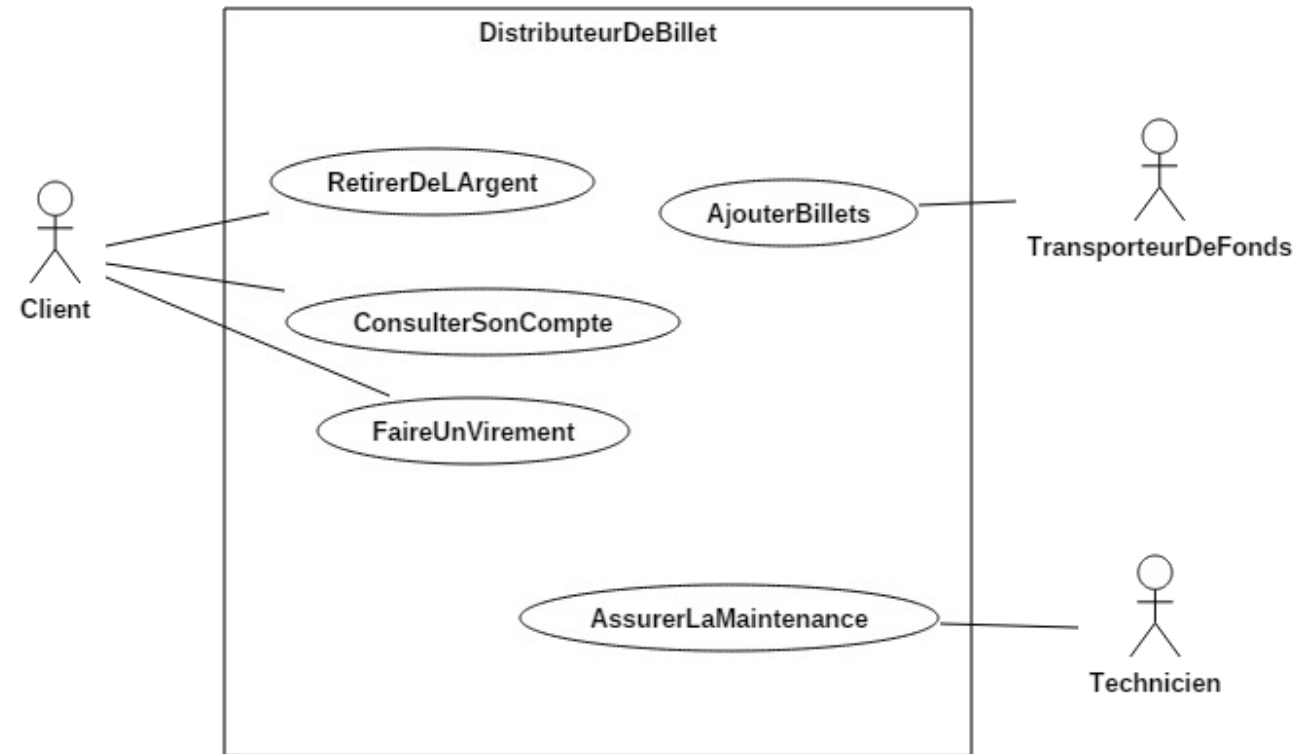
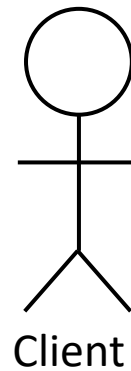


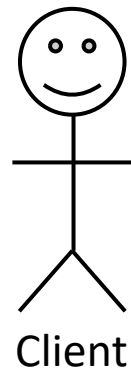
Diagramme de cas d'utilisation

- Un diagramme de cas d'utilisation décrit :
 - Les acteurs
 - Les cas d'utilisation
 - Le système
- Un modèle de cas d'utilisation ~~peut~~ **doit** être formé :
 - De descriptions textuelles
 - De diagrammes de séquences
 - De plusieurs sous-diagrammes de cas d'utilisation
 - ...

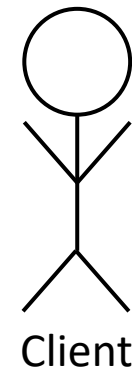
- Un acteur :
 - Est un élément externe du système qui interagit avec le système
 - Prend des décisions, des initiatives. Il est actif.
 - A un rôle qu'un utilisateur joue par rapport au système
- Notation graphique :



OUI



NON !



NON !

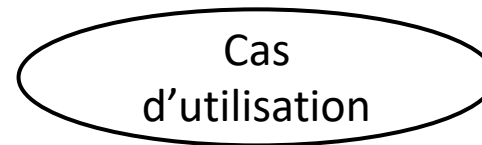
Acteur vs. Utilisateur



Ne pas confondre la notion d'acteur et de personne utilisant le système

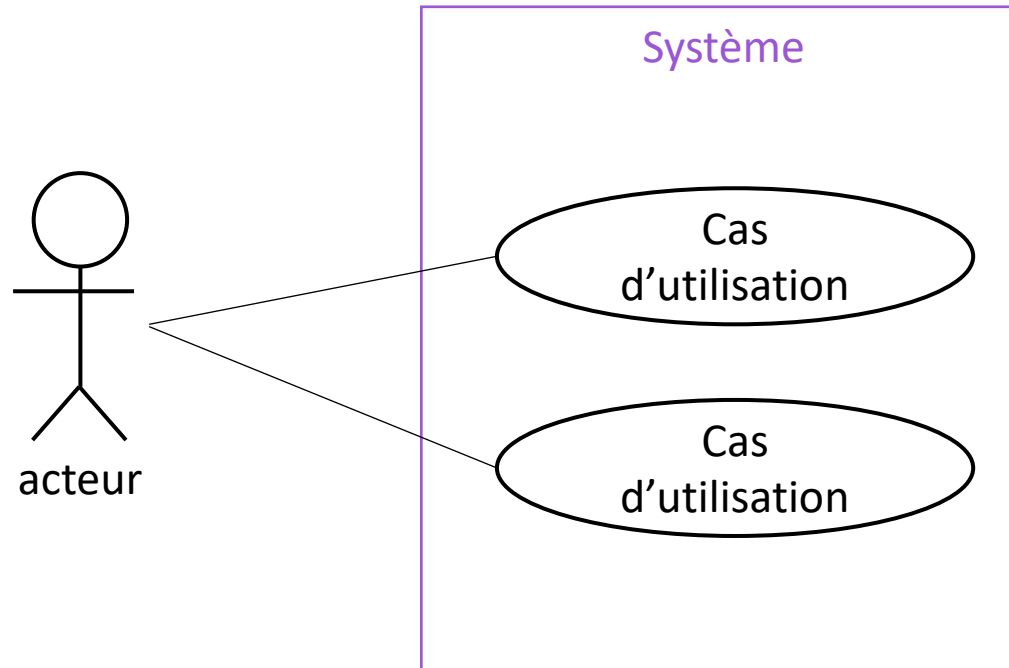
- Une même personne peut jouer plusieurs rôles
 - Exemple : Maurice est directeur mais peut jouer le rôle de guichetier
- Plusieurs personnes peuvent jouer un même rôle
 - Exemple : Paul et Pierre sont deux clients
- Un acteur n'est pas forcément un être humain
 - Exemple : Un distributeur de billets peut être un acteur

- Un cas d'utilisation :
 - Est une manière d'utiliser le système
 - Est une suite d'interactions entre un acteur et le système
- Correspond à une fonction **visible par l'acteur**
- Permet d'atteindre **un but** pour un acteur
- Doit être **utile en soi**
- Regroupe un **ensemble de scénarios** correspondant à un même but
- Notation graphique :



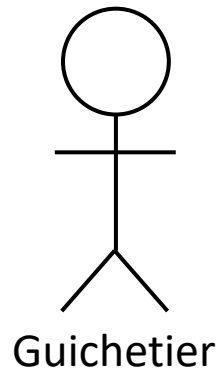
Le système

- Le système est l'ensemble des cas d'utilisation
- **Mais** ne contient pas les acteurs



- Tout diagramme de cas d'utilisation doit être accompagné d'informations textuelles.
- Pourquoi ?
 - Pour expliquer les cas d'utilisation
 - Pour préciser certains éléments qui ne peuvent pas être représentés graphiquement
 - Pour faciliter la communication avec le client de l'application

- Pour chaque acteur :
 - Choisir un identificateur représentatif de son rôle
 - Donner une brève description contextuelle. Doit répondre à la question « Qui est-ce ? », « Qu'est-ce qui est différenciant par rapport aux autres acteurs ? »



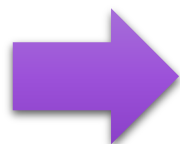
Un guichetier est **un employé de la banque** chargé de faire l'interface entre le système informatique et les clients qu'il reçoit au comptoir. Le guichetier peut réaliser des opérations courantes : création d'un compte, dépôt ou retrait d'argent.

- Pour chaque cas d'utilisation :
 - Choisir un identificateur représentatif (Exemples : CU120 ou UC430)
 - Donner une description textuelle simple (sous forme tabulaire)
 - La fonction réalisée doit être comprise de tous sous la forme d'un scénario nominal
 - Préciser ce que fait le système, ce que fait l'acteur
 - Préciser quelles sont les pré-conditions et/ou les post-conditions
 - Pré-conditions : conditions qui doivent être vraies avant que le cas d'utilisation puisse « démarrer »
 - Post-conditions : conditions qui doivent être vraies après l'exécution du cas d'utilisation
 - Préciser éventuellement les cas d'erreur
 - Un cas d'erreur correspond à une situation anormale qui empêche le déroulement normal du scénario nominal

Informations textuelles pour les cas d'utilisation

- Généralement sous la forme d'un tableau
- Exemple :

CUXXX - Nom du cas d'utilisation	
But	
Acteur	
Pré-conditions	
Scénario nominal	
Post-conditions	
Exceptions	

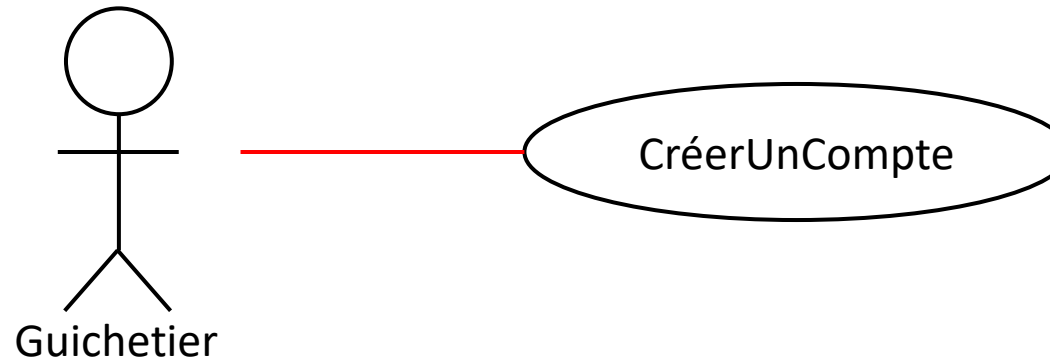


Les cas d'utilisation ne doivent pas se chevaucher !

Relation Acteur-Cas d'utilisation

- Une relation entre acteur et cas d'utilisation représente une communication initiée par l'acteur. Les messages échangés peuvent potentiellement être dans les deux sens.

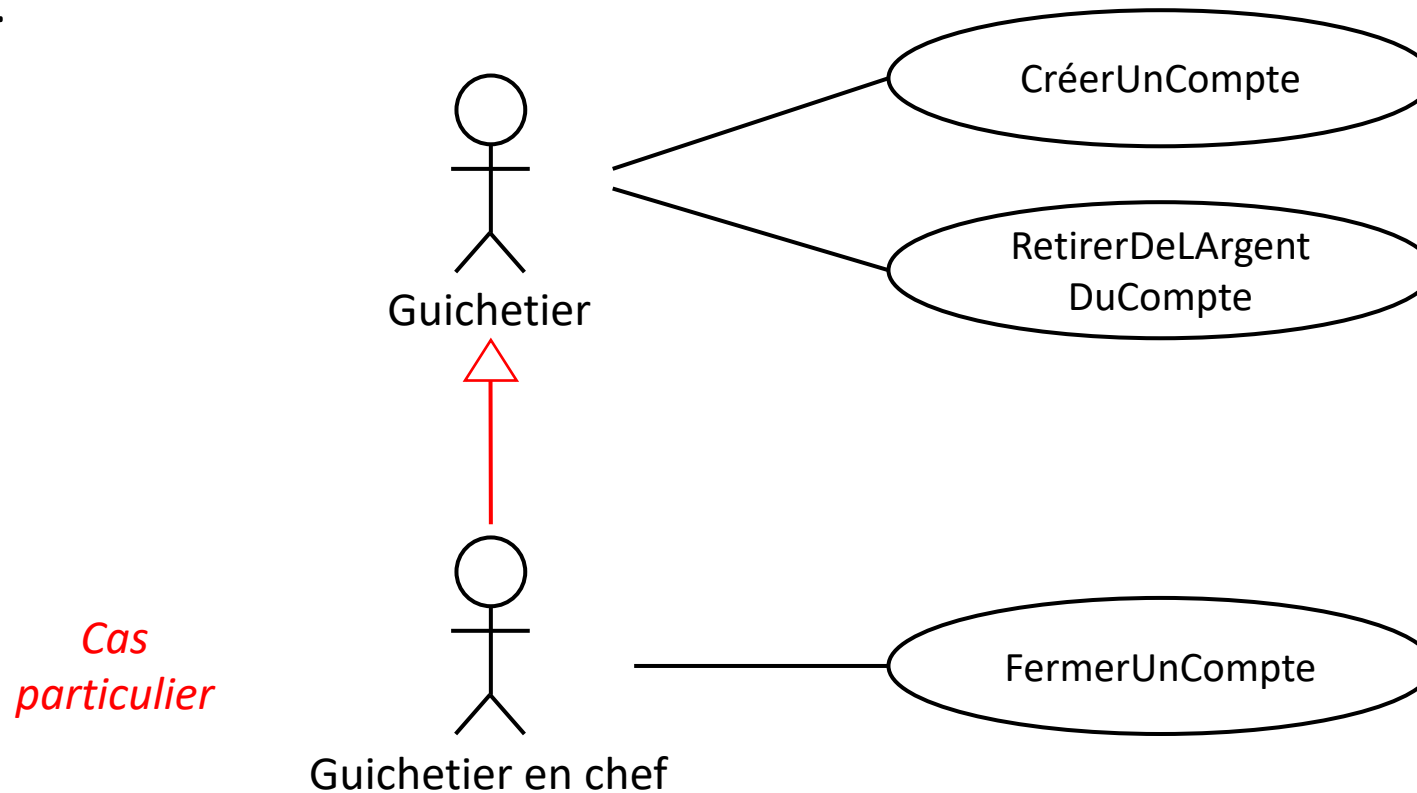
- Exemple :



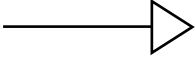
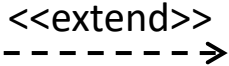
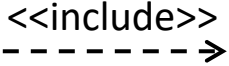
- La relation sera raffinée par la suite (*Spécifications externes*) :
 - Si l'acteur est humain : interface homme – système
 - Si l'acteur est un logiciel : interface logicielle

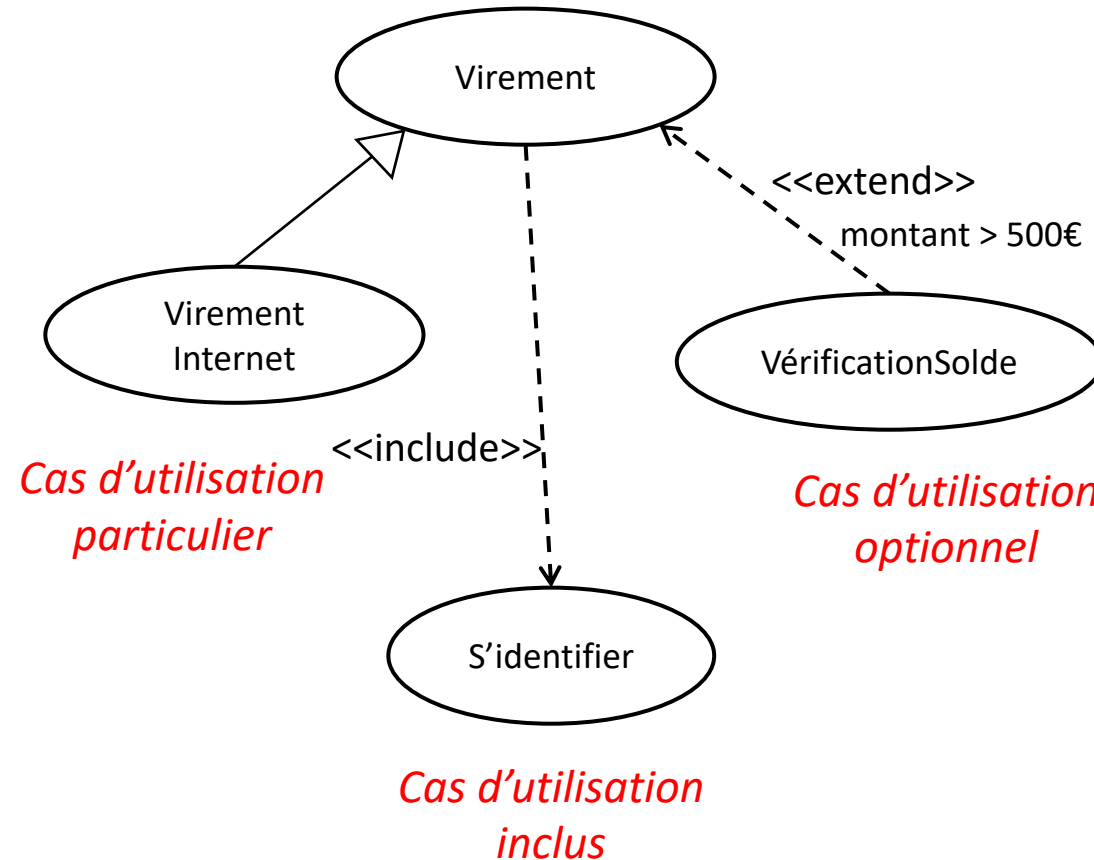
Relation entre acteurs

- La seule relation entre acteurs est la relation de **généralisation**
- Exemple :



Relations entre cas d'utilisation

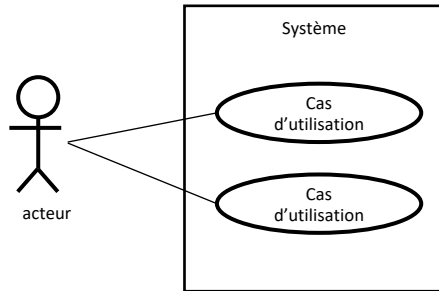
- Généralisation : 
 - À utiliser pour spécialiser un comportement général
- Extension : 
 - À utiliser quand il y a un comportement supplémentaire à ajouter pour le comportement défini dans un ou plusieurs cas d'utilisation
- Inclusion : 
 - À utiliser lorsqu'il y a des parties communes de comportement pour deux ou plus cas d'utilisation.



- Il n'y a pas de manière mécanique et totalement objective de repérer les cas d'utilisation :
 - Il faut bien déterminer les limites du système
 - Il faut se placer du point de vue de chaque acteur et déterminer comment il se sert du système
 - Il faut limiter le nombre de cas d'utilisation en choisissant le bon niveau d'abstraction
- Trouver le bon niveau de détail pour les cas d'utilisation est un problème difficile qui nécessite de l'expérience.

Extrait du dossier de spécifications – 1/2

1. Diagramme des cas d'utilisation général



- Description des acteurs
- Description des cas d'utilisation

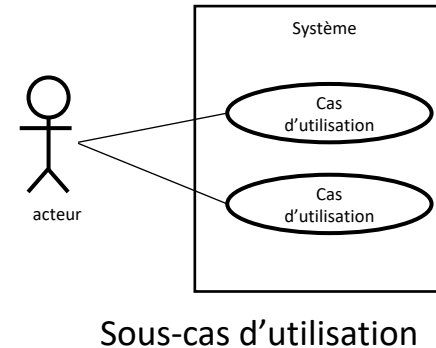
x nbUseCase

CU100 Cas d'utilisation	
But	
Acteur	
Pré-conditions	
Scénario nominal	
Post-conditions	
Exceptions	

+ diagramme de séquence

2. Description détaillée des sous- cas d'utilisation

x nbSousUseCase



CU100 Cas d'utilisation	
But	
Acteur	
Pré-conditions	
Scénario nominal	
Post-conditions	
Exceptions	

+ diagramme de séquence

3. Récapitulatif des cas d'utilisation

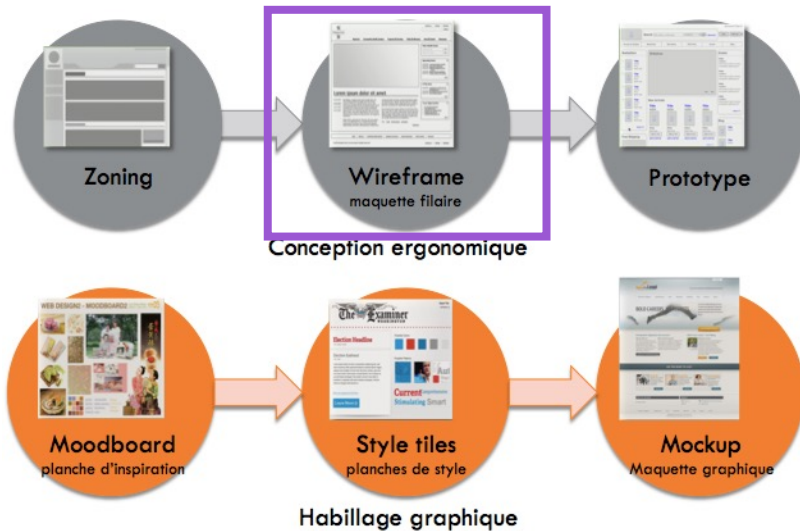
Identifiant	Tâche	Description	Priorité
CUXXX			Must Should May

Extrait du dossier de spécifications – 2/2

4. Spécifications des interfaces de communication

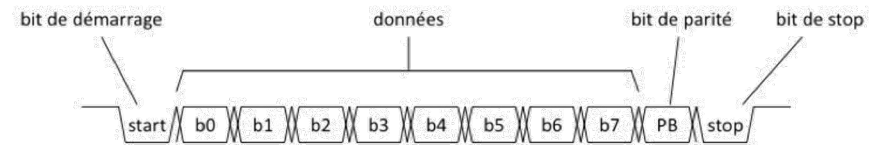
a. Interface Homme/Système (IHM)

Définition des wireframes



b. Interface logicielle (API)

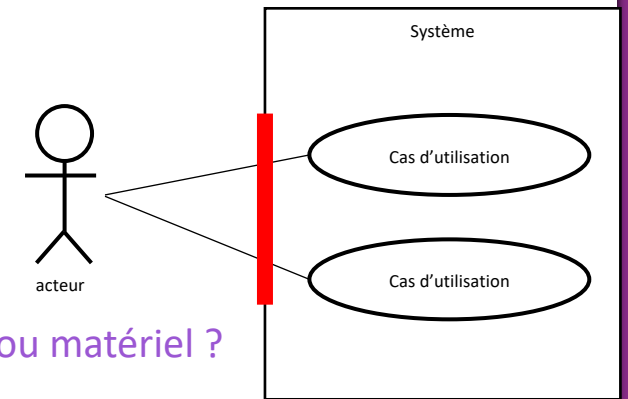
Définition de protocoles et interface de communication



```

"userID" : 1,
"isActive": false,
"identity":
{
  "lastName" : "Doe",
  "firstName" : "John",
  "birthDate" : "01/02/1975",
  "socialSecurityNumber" : "331-80-5651"
},
"address":
{
  "number" : "01",
  "street" : "2068 Thomas Street",
  "city" : "Roselle",
  "zipCode" : "60172",
  "state" : "IL"
},

```



Exercice : Station-service

- Les clients se servent de l'essence et le pompiste remplit les cuves.
- Le client se sert de l'essence de la façon suivante : il prend un pistolet accroché à une pompe et appuie sur la gâchette pour prendre de l'essence.
- Certains pompistes, appelés réparateurs, sont aussi qualifiés pour opérer des opérations de maintenance en plus des opérations habituelles des pompistes telles que le remplissage des réservoirs.

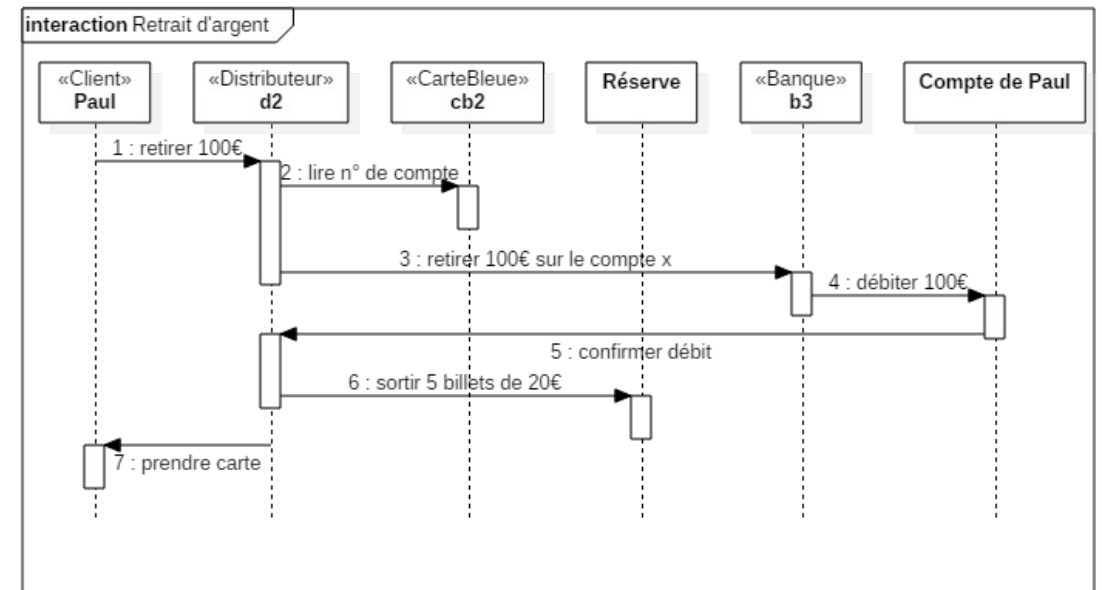


Diagramme de séquence

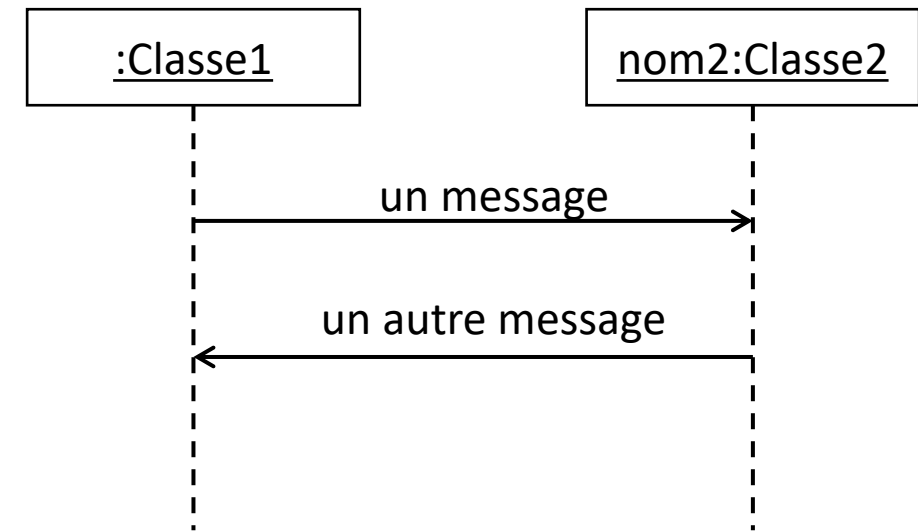
Modélisation des interactions

Diagramme de séquence

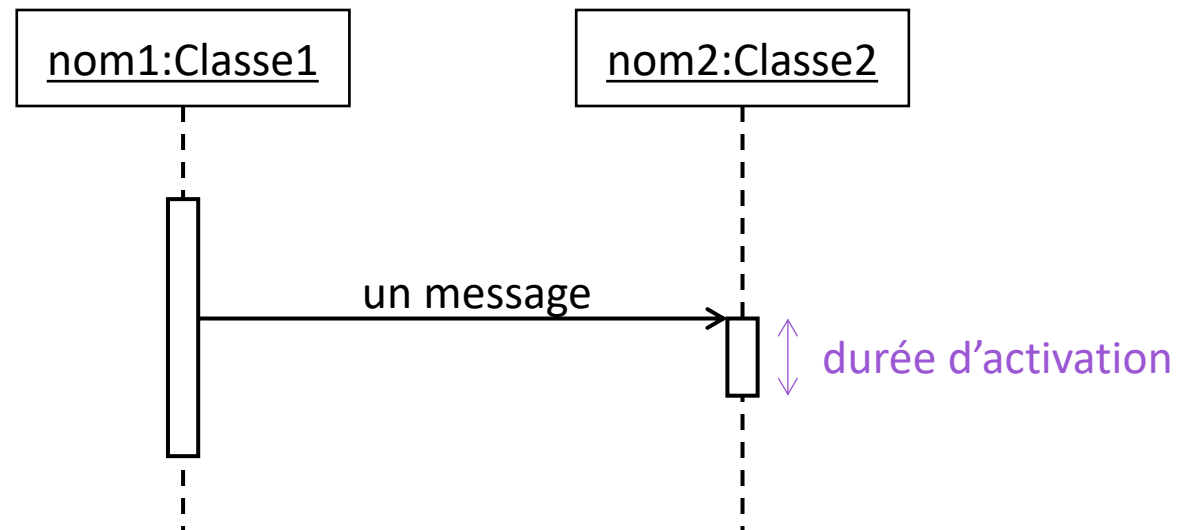
- Objectifs :
 - Décrire un scénario en explicitant les interactions entre objets/acteurs sous la forme d'envoi de messages
- Représentation des aspects dynamiques d'un système d'un point de vue temporel
- A utiliser pour :
 - La documentation des cas d'utilisation
 - La représentation des interactions entre objets



- Une interaction modélise un comportement dynamique entre objets. Elle se traduit par l'envoi de message entre objets en insistant sur la chronologie
- Représentation :
 - Un objet/acteur est matérialisé par un rectangle et une barre verticale, appelée ligne de vie
 - Les messages sont représentés par une flèche horizontale, orientées de l'émetteur vers le destinataire
 - La dimension verticale représente l'écoulement du temps



- Une période d'activité correspond au temps pendant lequel un objet effectue une action :
 - soit directement
 - soit par l'intermédiaire d'un autre objet qui lui sert de sous-traitant



- Au moins un scénario par cas d'utilisation
- Décomposer les interactions de grande taille en sous-tâches
 - Un diagramme de séquence par sous-tâche
- Un diagramme de séquence par condition d'erreur

Diagramme d'états transitions

Diagramme d'états transitions

- Objectif :
 - Modéliser l'évolution d'un objet suite à un événement extérieur
- Equivalent aux automates à états finis/machines à états

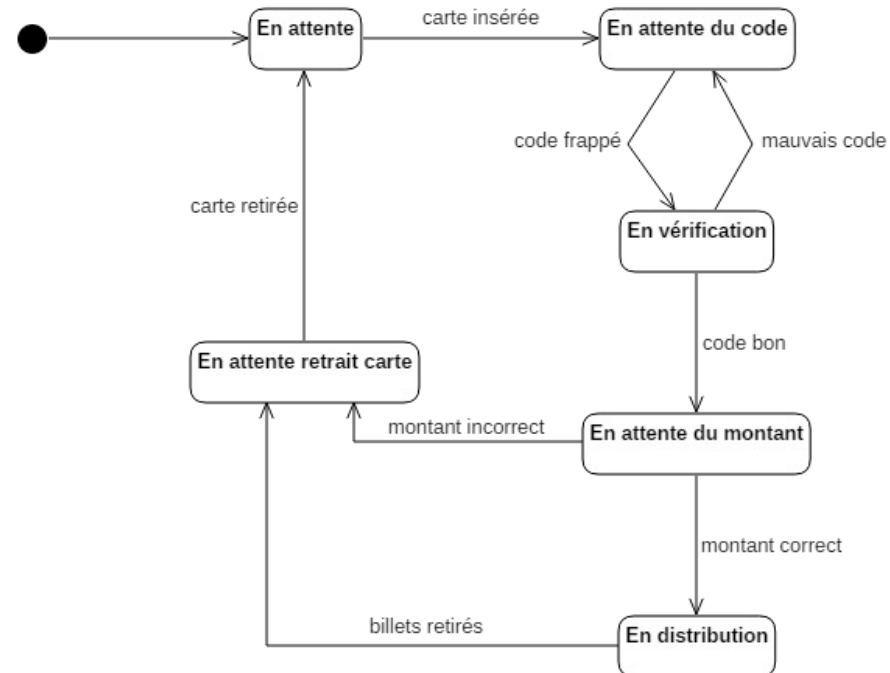
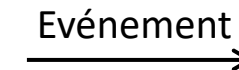


Diagramme d'états transitions

- Deux états particuliers :
 - Initial : représente le point à partir duquel l'objet existe
 - Final : représente l'activité finie
- Etat :
 - Représente l'état d'un objet
- Transition :
 - Représente le passage d'un état à un autre
 - Doit être annoté d'un label précisant l'événement qui déclenche la transition



- Modéliser avec un diagramme d'états-transitions le fonctionnement d'un téléphone.

